Schild TCB — Независимый технический обзор

Schild TCB — Независимый технический обзор и оценка безопасности

Статус документа: ознакомительный отчёт для сторонних инженеров **Точка зрения:** независимый аудитор (purple team: red + blue) **Дата:** 04.10.2026 **Гриф:** ограниченное распространение (автор — Ирганов А.Н.)

> Этот документ не является официальной оценкой соответствия. Он подготовлен «с нуля» как вводный обзор для инженеров, которые **впервые** видят проект, и отражает как объективные достижения, так и честно выявленные ограничения и остаточные риски. Стиль — purple team: мы смотрим одновременно глазами защитника (что уже сделано правильно) и атакующего (где остаются зазоры).

---

1. Резюме для руководства

Schild TCB — это приватный научно-исследовательский **прототип** доверенной вычислительной базы (Trusted Computing Base) и среды изолированной периферии, написанный на **Ada/SPARK 2022** (с элементами C и x86_64-ассемблера) поверх прекомпилированного микроядра **seL4**. Проект сочетает три сквозные темы:

1. **Формальная верификация** — ядро доказывается в SPARK на 4-м уровне строгости (`not-proved: 0` по 185 модулям), отдельно доказываются 7 демонов root-задачи (включая саму root-задачу, `SPARK_Mode => On`).
2. **Мандатное управление доступом (MLS)** — модель Bell-LaPadula с 5 уровнями допуска (Public…Top_Secret).
3. **Изоляция некритичной периферии** — сетевой стек и драйверы NVMe/e1000e выносятся в защитные домены микроядра, с контролем сбоев (crash-containment) и бюджетированием процессорного времени.

Проект существует в виде **двух образов** (важно не путать):

Образ	Сборка	Назначение	Где работает
`schild_os.iso`	`./build.sh` (default)	**bare-metal**: собственное ядро Ring 0, GOP-фреймбуфер, ECAM→NVMe→сеть, SMP	ThinkPad T580 ✅ · QEMU ✅
`sel4_primary.iso`	`SEL4_PRIMARY=1 … ./build.sh`	**верификационный**: seL4 (MCS) + Ada root-задача + изолированные серверы	QEMU ✅ · T580 ✅

Главное за последние итерации (октябрь 2026): (1) root-задача переведена на **MCS**-конфигурацию seL4 — ядерные CPU-бюджеты и жёсткая временная изоляция; (2) **seL4-стенд загружается на железе** (T580, 32 ГБ) — устранён root-cause зависания retype (7-МиБ boot-reuse из-за кварка `MAX_NUM_DRHU`); (3) **сетевая изоляция валидирована на железе** — seL4-NetServer раздаёт HTTP через реальную I219-V (ECAM-скан + MSI) в изолированном Ring 3.

Вывод аудитора (кратко). Это **зрелый исследовательский прототип**, а не готовый production-продукт. Автор проекта осознанно придерживается принципа «точных, покомпонентных утверждений» вместо широкого ярлыка «high-assurance»: что доказано — доказано, что проверено в QEMU — так и сказано, что валидировано на железе — помечено отдельно. С точки зрения security-инженера это правильная, редкая дисциплина. Основные остаточные риски — отсутствие IOMMU (VT-d, DMA-изоляция) и более слабая формальная верификация ветки x86_64 MCS относительно ARM.

---

2. Что такое Schild TCB и зачем он нужен

2.1. Понятие TCB

Trusted Computing Base (TCB) — минимальная совокупность кода и данных, которой мы вынуждены доверять, потому что её компрометация ставит под угрозу всю систему. Ключевая инженерная задача — сделать TCB **маленьким**, чтобы его можно было формально верифицировать, и **изолировать всё остальное** так, чтобы сбой или компрометация «периферии» (сетевой драйвер, файловая система) не могла нарушить гарантии безопасности ядра.

2.2. Три источника идей

Проект явно опирается на три проверенные парадигмы:

1. **ARINC 653** (авионика, Integrated Modular Avionics) — жёсткое **пространственное и временное** разделение партиций: у каждой партиции фиксированный временной слот (Major Time Frame), детерминированный обмен сообщениями и трехуровневый мониторинг здоровья (process / partition / module). Именно эти идеи реализованы пакетами `Schild_MTF`, `Schild_IPC_Ports`, `Schild_Scheduler` и монитором `Health_Monitor`.
2. **Junos-стиль** — изолированные демоны, транзакционный конфиг «commit confirmed» (`Schild_Config_Daemon`), разделение control-plane и data-plane.
3. **seL4** — микроядро с доказанной (формально) изоляцией, основанное на **capabilities**: каждый объект доступен только через явно переданный capability, а не через глобальное адресное пространство.

2.3. Чего проект добивается

- **Формально верифицированное ядро** (SPARK Level 4) — механически проверенные контракты, а не «верификация на глаз».
- **MLS (Mandatory Labeling Security)** — принудительная модель доступа по уровням допуска; нарушение правил Bell-LaPadula должно быть невозможным.
- **Изоляция сбоев** — сбой сетевого драйвера или сервера не должен ронять ядро (crash-containment + перезапуск домена через Health Monitoring).
- **Временная изоляция** — «прожорливый» или зависший процесс не должен забрать всё процессорное время у остальных (MCS-бюджеты).

---

3. Архитектура

3.1. Два исполняемых образа

Проект **двуликий**, и это надо понимать сразу:

	(`schild_os.iso`)	(`sel4_primary.iso`)
Роль	bare-metal, «настоящее» ядро	верификационный стенд
Ядро	собственное (Ring 0, boot.S → Long Mode)	seL4 (MCS), прекомпилированное
Слои	4 слоя Ada/SPARK	Ada root-задача + 3 C-сервера
Железо	T580 + QEMU	только QEMU (нет COM-порта на T580)
Назначение	сеть/NVMe/фреймбуфер/SMP	формальная верификация + домены

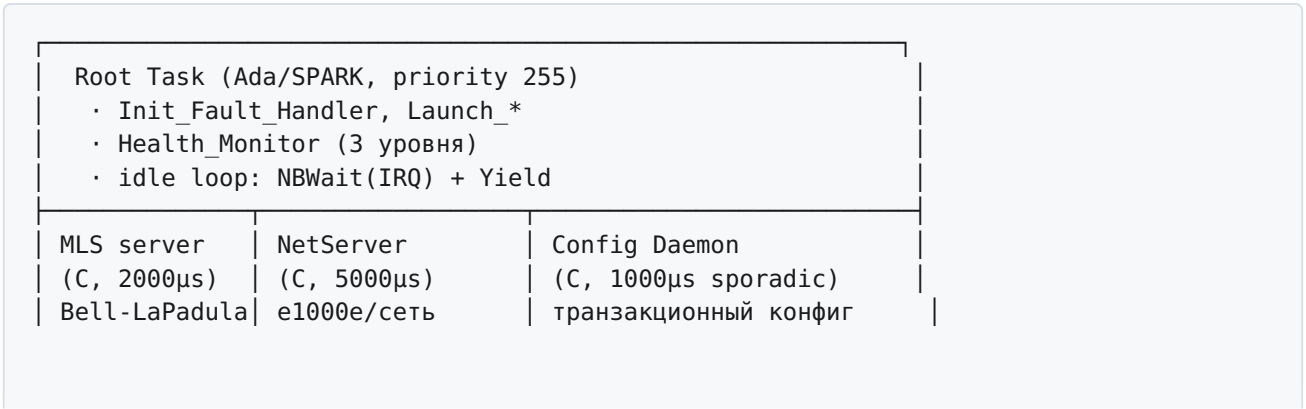
Почему так. Автономное ядро демонстрирует полный вертикальный стек (загрузка → NVMe → сеть → HTTP), а seL4-конфигурация — это «чистый» стенд, где проверяются гарантии изоляции и верификации, без зависимости от драйверов.

3.2. Автономное ядро: четыре слоя

Слой	Содержимое
`01_hardware`	MMU, APIC/IOAPIC, NVMe, e1000e, SMP, PIT-таймер
`02_security`	MLS Bell-LaPadula, cap-space, IPC-шлюз, файрвол
`03_crypto`	AES-NI, SHA-256, HMAC-SHA256 (FIPS 198-1), ГПСЧ, ECC, RSA
`04_userland`	SchildFS/VFS, сокеты, сетевой стек (ARP/IPv4/ICMP/TCP/HTTP)

Плюс двухуровневый планировщик: **Major Time Frame** (статическая циклическая матрица партиций) выбирает партицию → приоритет выбирает задачу внутри партиции.

3.3. seL4-конфигурация: root-задача и домены



```
└──────────────┴──────────────────┴──────────────────────────────┘
              сериализация через IPC (seL4 endpoints)
```

Каждый домен имеет:

- **TCB** (поток) и **CSpace/VSpace** (собственное или разделяемое адресное пространство возможностей);
- **Scheduling Context (SC)** — ядерный CPU-бюджет (budget/period в мкс);
- **Reply-объект** — контейнер для разовых reply-capabilities при IPC-Call;
- **fault-эндпоинт** (куда ядро шлёт сообщение о сбое) и **timeout-эндпоинт** (куда шлёт сообщение при исчерпании бюджета).

3.4. Ключевой момент: MCS-миграция

В недавней итерации root-задача переведена с **non-MCS** на **MCS** (Mixed-Criticality Systems) API seL4. Практический смысл:

- **раньше** — планировщик лишь чередовал потоки (кооперативно), гарантий бюджета не было;
- **теперь** — у каждого домена есть **ядерный CPU-бюджет**: исчерпал бюджет → поток вытесняется, а при спорадическом бюджете (budget < period) — ещё и блокируется до следующего периода, с доставкой **timeout-сбоя** в timeout-эндпоинт.

Именно это превращает «детерминированный порядок» (ARINC 653 MTF на уровне пользователя) в **жёсткую временную изоляцию** (ядерные бюджеты).

---

4. Модель безопасности

4.1. MLS (Mandatory Labeling Security)

Реализована модель **Bell-LaPadula**: у каждого процесса — уровень допуска (`Security_Level`), у объектов — гриф секретности. Два классических правила:

- **no read up** — процесс не может читать данные выше своего допуска;
- **no write down** — процесс не может писать в объекты ниже своего допуска (иначе произошла бы утечка вниз).

В seL4-конфигурации уровней **5** (Public, Internal, Confidential, Secret, Top_Secret), в автономном ядре — **4**. Политика вынесена в SPARK-доказанные пакеты (`Schild_MLS_Policy`, `Schild_Config_Daemon`), а не в неверифицированный C-код: решение Bell-LaPadula принимает доказанный Ada-гейт.

4.2. Capabilities (seL4)

В seL4-конфигурации доступ к объектам — только через **capabilities** (дескрипторы прав). Нет глобального «корня»; чтобы другой домен мог обратиться к эндпоинту или

странице памяти, ему надо явно «минтануть» (mint) копию capability с ограниченными правами и (опционально) бейджем. Это даёт тонкую настройку прав: например, эндпоинт сбоя NetServer'а минтится с бейджем `0xE2`, чтобы root-задача по бейджу понимала, **какой** домен упал.

4.3. Изоляция сбоев (Health Monitoring, ARINC 653)

Когда поток делает запрещённое (обращение к несмапленной памяти, #PF, неверный capability), seL4 **не роняет систему**, а шлёт сообщение о сбое в fault-эндпоинт домена. Root-задача ловит его и применяет политику **трехуровневого** мониторинга здоровья:

Уровень	Действие	Когда
process	RESTART (перезапуск потока)	первые N сбоев
partition	STOP (домен остаётся приостановленным)	повторные сбои (системная проблема)
module	IDLE (все партиции остановлены)	исчерпан лимит всех доменов

Плюс отдельный канал **timeout** (при исчерпании CPU-бюджета спорадического домена), который root-задача различает по бейджу scheduling-контекста.

4.4. Детерминированный IPC (ARINC 653)

Реализованы два классических порта ARINC 653:

- **Sampling Port** — хранит только последнее значение (перезапись);
- **Queuing Port** — ограниченная SPSC-FIFO (производитель/потребитель).

Оба SPARK-доказаны (инвариант типа, flow, отсутствие гонок) и покрыты known-answer тестами.

---

5. Текущее состояние (на 04.10.2026)

5.1. Реализовано и валидировано

Возможность	Статус	Где проверено
Автономное ядро (4 слоя, SMP, NVMe, e1000e, сеть, HTTP)	✅	T580 + QEMU
seL4 root-задача (MCS) + 3 домена	✅	T580 + QEMU
Изоляция сети в Ring 3 (NetServer → I219-V, ECAM+MSI)	✅	T580
Изоляция хранилища в Ring 3 (NVMe + SchildFS → NetServer)	✅	QEMU (20/20)
Per-partition CPU-бюджеты (NetServer 5000, MLS 2000, Config 1000 µs)	✅	QEMU
Timeout-механизм (TCBSetTimeoutEndpoint + SC-бейджи)	✅	QEMU
Crash-containment + HM-restart всех 3 доменов	✅	QEMU
3-уровневый HM (process→partition→module)	✅	QEMU
Time Partitioning (MTF), двухуровневый планировщик	✅	KAT + QEMU
Детерминированный IPC (sampling/queuing)	✅	SPARK + KAT
SchildFS (GPT-gap, LBA 2016)	✅	T580 (write-path)
Крипто-примитивы (AES-NI, SHA-256, HMAC, ГПСЧ, ECC, RSA)	✅	KAT

5.2. Верификация

- **SPARK Level 4, автономное ядро:** `prove_kernel.sh` → **185 модулей,** `not-proved: 0` (0 непроверенных проверочных условий).
- **SPARK Level 4, root-задача:** `verify.sh` (`verify.gpr`) → **7 модулей** (MLS-политика, Config Daemon, Audit Logger, Boot Manager, HM Model, Launch Model, **Root Task**) —

все VC доказаны. Драйвер `schild_root_task.adb` переведён `SPARK_Mode => On` и доказан (54 checks).

- **Регрессионные маркеры QEMU:** `run_tests.sh` → **20 маркеров PASS** (MLS, NetServer, IRQ, DMA, 3×fault-containment, timeout, эскалация HM, NVMe-init, SchildFS write-path).
- **Сборка:** 181 объект, 0 ошибок.

5.3. Что НЕ является «доказанным» (важно для аудитора)

- `sel4_primary.iso` — загружается на T580 (32 ГБ) с 2026-10-04 (фреймбуферная консоль); сетевая изоляция seL4-NetServer валидирована на железе (I219-V, MSI).
- seL4-биндинги (`schild_sel4_syscall.*`): спецификация — `SPARK_Mode => On` (типы и операции SPARK-видимы, контракты `Global => null`), **тело** — `Off` (retype/map/MMIO/MSI — доверенная аксиоматика, не доказывается). Драйвер `schild_root_task.adb` теперь `On` и доказан (54 checks); до W4 он был `Off`.
- x86_64 MCS ядро seL4 формально верифицировано **слабее**, чем ARM MCS (официальный статус upstream seL4).

---

6. Оценка безопасности (purple team)

6.1. Что сделано правильно (сильные стороны)

1. **Честная модель assurance.** Проект принципиально отказывается от широкого ярлыка «high-assurance»: каждый компонент заявляет ровно тот уровень гарантий, который реально достигнут (SPARK Level 4 — для доказанных модулей, QEMU-валидация — для интеграционных сценариев, аппаратная валидация — отдельно). Это редкая и правильная дисциплина, которая снижает риск «перепродажи» гарантий.
2. **Политика безопасности в доказанном коде.** Решение Bell-LaPadula и валидация конфигурации принимаются SPARK-доказанными пакетами, а не неверифицированным C-кодом на месте.
3. **Изоляция сбоев — не на бумаге.** Crash-containment и перезапуск домена продемонстрированы в QEMU **для всех трёх доменов**, включая эскалацию RESTART → STOP при повторных сбоях и детекцию исчерпания CPU-бюджета.
4. **Принцип наименьших прав.** Фальт- и timeout-эндпоинты бейджируются и mint’ятся с ограниченными правами — root-задача различает домены, не имея лишних прав.

6.2. Найденные и исправленные дефекты (в ходе аудита)

#	Дефект	Влияние	Статус
1	**ReadOnly-право = 1 вместо 2** (`RO := 1` = write-бит в раскладке CapRights)	«потребительские» страницы MLS-колец маппились как write-only и ядро маскировало их в «нет доступа» вместо read-only	✅ исправлено (`RO := 2`)
2	Карта памяти обрывалась на 50 записях (все device)	`Total RAM` показывал `0 MB`	✅ исправлено (490 MB)
3	Fault-обработчики MLS/Config были notification вместо endpoint	латентный дефект (notification fault handler)	✅ исправлено ранее (`4f87475`)
4	MCS-ядро собрано без `CONFIG_PRINTING`	ядро молчало в QEMU (пустой serial)	✅ пересобрано с PRINTING
5	`Rollback(CD)` при `Mode=Validating` (контракт требует `Tentative`)	демо-баг, вскрыт доказательством root-задачи (W4)	✅ исправлено
6	`Wait(Fault_Slot)` при `Fault_Slot=0` (фолбэк-путь init fault handler)	containment-тест мог блокироваться на `Wait(0)`	✅ исправлено (guard `Fault_Slot>0`)
7	`C_Addr` NVMe = `schild_ptr_to_u64` (виртуальный адрес) вместо `schild_dma_phys`	PRP/очереди DMA писались бы по виртуальному адресу в Ring 3	✅ исправлено (W7)
8	NIC и NVMe в одном device-untyped; NVMe-retype из «съеденного» untyped	retype `0xA001` (`RangeError_InvalidRoot`)	✅ исправлено (retype из остатка `Dev_UT`)
9	Повторный `Map_Ecam` в NVMe-discovery	«ECAM unavailable» (ECAM уже замаплен NIC-путём)	✅ исправлено (переиспользование)

6.3. Остаточные риски (surface атаки, на что смотреть red team)

1. **Нет IOMMU (VT-d).** DMA-изоляция не реализована: скомпрометированная периферия (PCIe-устройство) теоретически может читать/писать произвольную физическую память, обходя VSpace. Это **главный** пробел в «изоляции периферии», заявленной как цель проекта.

2. **x86_64 MCS верифицировано слабее ARM MCS.** Формальные гарантии seL4 на этой ветке ниже, чем на референсной ARM-ветке; это следует явно отражать в любом security-утверждении.
3. **seL4-механизм (syscall-тело) вне доказательства.** Спецификация `schild_sel4_syscall.ads` — `SPARK_Mode => On` (типы SPARK-видимы, контракты `Global => null`), сама root-задача `schild_root_task.adb` — `On` (доказана, 54 checks, W4). Вне доказательства остаётся только **тело** syscall-прослойки (retype/map/MMIO/MSI — доверенная аксиоматика). Логика эскалации — доказанный `Schild_HM_Model`, запуск «ровно раз» — `Schild_Launch_Model` (оба Level 4).
4. **e1000e IRQ-driven RX — ✅ валидирован на железе для обоих стеков.** (а) Автономное ядро: RX переведён с busy-poll на прерывание (вектор 0x30 → `e1000e_irq_handler` → флаг → `hlt`); на T580 подтверждён путь `[NET] RX IPv4` → `[TCP] SYN→SYN-ACK` → `[HTTP] reply sent`. (б) seL4-NetServer: root-задача находит I219-V через ECAM, программирует MSI (badge `0xE1`), мапит BAR0 (`0xE8200000`) — end-to-end TCP/HTTP через изолированный Ring 3 процесс.
5. **Спектр аппаратных атак** (rowhammer, SMM, DMA через легаси-устройства, meltdown/spectre-класс) — вне текущей модели. Аппаратные контрмеры ядра частично активны: **CR0.WP ✅, SMEP ✅, IBRS (Spectre v2) ✅**; **SMAP ❌** — выключен, т.к. `CONFIG_PRINTING=1` (нужен для boot-логов) в этой версии seL4 явно гейтит SMAP. VT-x выключен (verified-конфиг).
6. **Одиночная точка доверия — root-задача.** Она держит все capability; её компрометация эквивалентна компрометации всей seL4-конфигурации (хотя это и стандартно для seL4-стендов).

6.4. Общий вывод purple team

Архитектурно проект **выстроен правильно** и честно: изоляция через capabilities, политика в доказанном коде, сбои локализуются и восстанавливаются, время бюджетируется на уровне ядра. Это сильная база для исследовательского прототипа. Основные зазоры — **не отсутствие дисциплины, а незакрытые аппаратные уровни** (IOMMU/DMA, аппаратные атаки) и **граница формального доказательства** (x86_64 MCS, сама root-задача). Для перехода от прототипа к чему-то более серьёзному именно они должны закрываться в первую очередь.

---

7. Векторы развития

1. **IOMMU (VT-d)** — DMA-изоляция устройств: `-device intel-iommu` в QEMU, `CONFIG_IOMMU` в ядре, настройка IO-пространства в root-задаче (retype IOASID/IOPT, маппинг DMA-регионов, привязка устройств). **Статус:** частично / экспериментально — ядро собирается с IOMMU и детектирует VT-d; сдвиг X86-лейблов инвокаций под `CONFIG_IOMMU` исправлен (root-задача проходит все 18 маркеров под IOMMU-ядром). Блокер: под `KernelVerificationBuild=OFF` спорадический SC зависает недетерминированно. Диагностика локализовала триггер — race/timing в sporadic

refill-пути (budget<period) ядра: round-robin стабилен (но ломает B.6-таймаут), sporadic 100ms ≈33% зависонов, sporadic 10ms ≈67% (меньше период — хуже); конфигурационного обхода нет. IOMMU помечен **experimental/flaky, не PASS**; DMA-маппирование (IOPT + привязка устройств) отложено до стабилизации upstream x86_64 MCS.

2. **Усиление формального доказательства** — ✅ частично: логика эскалации `Health_Monitor` доказана (`Schild_HM_Model`, Level 4). Осталось: доказать `Launch_*`-последовательности, перенести саму root-задачу из `SPARK_Mode => Off`, отслеживание upstream-доказательства x86_64 MCS.
3. **Аппаратная валидация e1000e RX** — ✅ закрыто для обоих стеков: автономное ядро (MSI → hlt-idle) и seL4-NetServer (ECAM-скан I219-V + MSI, IRQ-RX badge `0xE1`; HTTP отдаётся через изолированный Ring 3 процесс).
4. **Аппаратные контрмеры** — SMEP/SMAP, VT-x для вложенной изоляции, оценка meltdown/spectre-класса.
5. **Завершение ARINC 653** — расширение MTF на реальное число партиций, интеграция временного разделения в автономное ядро (не только seL4-стенд).

---

8. Как воспроизвести (для инженеров)

Окружение: Linux x86_64, GCC/GNAT 14.2 (Alire toolchain), QEMU, grub-mkrescue, ninja, gnatprove (Alire release).

```
cd src/schild_core

# 1) Автономное ядро (bare-metal)
./build.sh                                # → schild_os.iso

# 2) seL4-конфигурация (MCS) — ядро + root-задача
KERNEL_MCS=ON scripts/build_sel4_kernel.sh      # → tools/sel4/kernel.elf
SEL4_PRIMARY=1 ./build.sh   # → sel4_primary.iso

# 3) Регрессионный прогон в QEMU (20 маркеров)
SKIP_BUILD=1 SEL4_PRIMARY=1 ./scripts/run_tests.sh

# 4) Верификация
bash scripts/verify.sh                    # 7 модулей root-задачи (Level 4)
bash scripts/prove_kernel.sh              # автономное ядро (185 модулей, not-proved:
0)
```

Ключевые артефакты:

- `src/schild_core/tools/sel4/` — root-задача seL4 (Ada + C + ASM);
- `src/schild_core/src/` — автономное ядро (4 слоя);
- `src/schild_core/scripts/` — сборка/верификация/тесты.

9. Где читать подробнее

Документ	Содержание
`docs/technical/ASSURANCE.md`	assurance case, модель доверия, roadmap верификации
`docs/technical/ARCHITECTURE.md`	архитектура и syscall ABI
`docs/technical/MCS_MIGRATION.md`	миграция на MCS: labels, ABI, шаги
`docs/technical/SEL4_INTEGRATION.md`	лог интеграции seL4 (Stage 5–8)
`docs/technical/SECURITY.md`	аудит уязвимостей (2026-08-11)
`docs/technical/CHANGELOG.md`	только подтверждённые результаты
`PROJECT_PASSPORT.md` / `README.md`	паспорт и описание программы

---

10. Заключение

Schild TCB — **архитектурно грамотный, честно задокументированный** исследовательский прототип TCB на Ada/SPARK поверх seL4. Он демонстрирует работающую связку: формальная верификация (SPARK Level 4), мандатное управление доступом (MLS Bell-LaPadula), изоляция и восстановление после сбоев (crash-containment + трехуровневый Health Monitoring) и жёсткая временная изоляция (MCS CPU-бюджеты). Для инженера, впервые знакомящегося с проектом, главное — понимать его **двуликость** (bare-metal ядро vs seL4-стенд) и **границы гарантий** (что доказано, что проверено в QEMU, что — только на железе).

Рекомендуемый фокус следующей итерации — закрыть оставшиеся зазоры: **IOMMU (VT-d)** для честной DMA-изоляции (сейчас — изоляция процессов без изоляции DMA) и **усиление формального доказательства** root-задачи/ветки x86_64 MCS. Сетевая изоляция (seL4-NetServer на железе) уже достигнута; после IOMMU и усиления доказательства можно всерьёз обсуждать применение прототипа за пределами стенда.
